PharmaOnto

Rapport technique et guide de validation finale

Web Sémantique · ESPRIT · 5 TWIN · Année universitaire 2026–2027

Ahmed Saidani, Fedi Ben Khalifa et Farah Maiti
Version 2.0 · 29 octobre 2026

PharmaOnto est un démonstrateur local qui transforme un corpus pharmaceutique documenté en connaissances interrogeables. L'application relie médicaments, substances actives, effets et contre-indications à cinq patients fictifs. Elle montre les inférences, les requêtes SPARQL et les sources utilisées pour produire une réponse.

La livraison comprend le modèle OWL 2 DL, son extension SWRL, les données structurées, les scripts de reconstruction, l'application Django, Fuseki, les tests et la présentation. La campagne finale comprend **51 tests réussis**, dont la comparaison des 12 requêtes entre RDFLib et un serveur Fuseki réel.

Correspondance avec la consigne

Exigence du sujet	Réalisation et preuve
Modélisation sous Protégé	pharmaonto.owl, 32 classes et 15 relations d'objet
Validation HermiT	Profil OWL 2 DL accepté, ontologie cohérente, 0 classe insatisfiable
Intégration de données	Corpus JSON sourcé transformé en individus OWL par RDFLib
Interface web	Django et interface française avec catalogue, patients, graphe et requêtes
Fuseki et SPARQL	Graphe matérialisé, réponses JSON et 12 requêtes SELECT
Recherche en langage naturel	spaCy local, entités et intentions contrôlées, traduction vers SPARQL
Intégration sur une machine	INSTALLER.cmd, DEMARRER.cmd et environnement .venv

L'équipe de trois membres a été confirmée comme approuvée. Les noms sont conservés tels qu'ils figurent dans le brouillon. Le projet utilise des données françaises et des patients fictifs. Il ne prétend pas fournir une recommandation de traitement ou une validation clinique.

Installation et lancement

Préparer la machine de soutenance

Extraire complètement le ZIP. Installer Python 3.11 et un JDK 17 ou plus récent si nécessaire. Vérifier les commandes ci-dessous dans un terminal. Le JDK fournit **javac**, nécessaire à la validation HermiT. Protégé doit être installé séparément pour examiner l'ontologie dans son interface.

```
python --version
java -version
javac -version
```

Exécuter **INSTALLER.cmd** avec Internet pour créer .venv et installer les paquets Python. Fuseki 5.5.0 est inclus et vérifié par SHA-512. Sous Windows avec Python 3.11, les dépendances exactes de la campagne sont dans requirements-lock-windows-py311.txt. Une installation sur une autre machine doit recréer .venv, pas le copier.

Lancer la démonstration

Exécuter **DEMARRER.cmd**. Le lanceur démarre Fuseki sur 127.0.0.1:3030, puis Django sur 127.0.0.1:8000. Il attend un état de santé valide avant d'ouvrir le navigateur. L'indicateur doit afficher **Fuseki connecté**. Conserver la console ouverte et utiliser Ctrl+C pour arrêter les deux processus.

Après installation, la recherche fonctionne sans Internet. Les liens de consultation des RCP restent des liens web. Les ports 3030 et 8000 doivent être libres. Le lanceur ne remplace pas un service déjà en cours d'exécution.

Fichiers de travail

Dossier ou fichier	Usage
ontology/pharmaonto.owl	Version source OWL 2 DL pour Protégé
ontology/pharmaonto-swrl.owl	Version autonome incluant la règle SWRL
ontology/pharmaonto-materialized.ttl	Graphe chargé par Fuseki
data/corpus.json	Données de référence modifiables
queries/	12 SELECT et 2 CONSTRUCT exécutables
validation/	Résultats HermiT, SHACL, pytest et comparaison Fuseki
docs/	Slides, rapport et présent guide

Dépannage limité à la démonstration

Si Java est introuvable, corriger JAVA_HOME ou le PATH et rouvrir le terminal. Si Fuseki ne démarre pas, lire les messages de sa console. DEMARRER_LOCAL.cmd propose un mode RDFLib explicite avec le même graphe pour le diagnostic. Ce mode ne remplace pas la démonstration Fuseki attendue. Le serveur Django utilisé est destiné à l'exécution locale, sans déploiement public.

Modèle et corrections apportées

Les sept concepts principaux sont Medicament, PrincipeActif, EffetSecondaire, ContreIndication, Patient, Pathologie et ObservationEffet. Ils sont disjoints. Le nom Pathologie est conservé pour la continuité avec le brouillon, avec un sens explicite plus large d'état clinique. Le corpus comprend 37 individus nommés, tous déclarés distincts.

Groupe	Individus
Spécialités pharmaceutiques	6
Principes actifs	4
États cliniques	7
Concepts d'effets possibles	4
Contre-indications	3
Associations médicament-effet	8
Patients fictifs	5

Axiomes essentiels

Les classes Princeps et Generique sont disjointes. Chaque Generique est lié à exactement un Princeps par estGeneriqueDe, propriété fonctionnelle. Cette contrainte est un choix du corpus : elle n'a pas vocation à couvrir tous les contextes réglementaires. Les classes SurOrdonnance et SansOrdonnance expriment le statut français au moment de la consultation.

MedicamentAntiInflammatoire est équivalente à Medicament et contientPrincipeActif some AntiInflammatoire. Des définitions analogues existent pour les antibiotiques et antidiabétiques. Les catégories de substances peuvent se recouvrir : l'ibuprofène est aussi un analgésique. PersonneAgee est une sous-classe d'Adulte. Les âges sont déclarés et contrôlés par SHACL, sans prétendre à une classification automatique par âge.

Corrections du brouillon

Les exemples génériques non identifiés ont été remplacés par AMOXICILLINE BGR et METFORMINE BGR avec leurs codes CIS et sources. Les formes exactes ont été corrigées. La masse dosée indique le sel lorsque nécessaire. Le modèle ne reconduit pas de statut princeps non documenté pour ADVIL ou COUMADINE, ni les valeurs de remboursement et codes CIM-10 insuffisamment cadrés du brouillon.

La fréquence n'est plus attachée globalement à un effet. Une ObservationEffet lie une spécialité à un effet et à sa fréquence documentée. Ainsi, les troubles digestifs sous GLUCOPHAGE sont très fréquents dans le RCP, tandis qu'aucune fréquence universelle n'est déduite pour tous les médicaments. La gravité et la fréquence restent deux dimensions distinctes.

La modélisation de dosage est bornée aux six spécialités mono-substance du corpus. Une extension aux associations fixes nécessiterait de réifier chaque composition et chaque quantité. Les indications et contre-indications sélectionnées ne constituent pas un inventaire complet des RCP.

Dictionnaire des relations

Propriété d'objet	Domaine	Valeurs visées
contientPrincipeActif	Medicament	PrincipeActif
traite	Medicament	Pathologie
estTraitePar	Pathologie	Medicament
provoque	Medicament	EffetSecondaire
aPourContreIndication	Medicament	ContreIndication
concerne	ContreIndication	Pathologie
interagitAvec	PrincipeActif	PrincipeActif
estGeneriqueDe	Generique	Princeps
souffreDe	Patient	Pathologie
prend	Patient	Medicament
estContreIndiquePour	Medicament	Patient
aObservationEffet	Medicament	ObservationEffet
effetObserve	ObservationEffet	EffetSecondaire
observationDe	ObservationEffet	Medicament
interactionMedicamenteuseAvec	Medicament	Medicament

Symétrie : interagitAvec et interactionMedicamenteuseAvec. **Inverses :** traite / estTraitePar et aObservationEffet / observationDe. **Chaîne :** aObservationEffet puis effetObserve implique provoque. Le libellé français de provoque est « peut provoquer », et n'exprime pas un effet systématique.

Propriétés de données

Huit propriétés fonctionnelles sont utilisées : nomCommercial, dosageMg, substanceDosee, formeGalenique et codeCIS pour Medicament ; frequence pour ObservationEffet ; description pour ContreIndication ; age pour Patient. Les types sont xsd:string, xsd:decimal et xsd:integer selon le champ. Les annotations dcterms:source et dcterms:date conservent la provenance. po:donneeFictive marque les patients.

Les domaines et codomaines OWL participent au raisonnement : ils peuvent produire des types. Ils ne doivent pas être présentés comme de simples règles de formulaire. Les contrôles de présence et de forme attendue relèvent du fichier shapes.ttl.

Raisonnement et validation des données

Inférences OWL

HermiT lit pharmaonto.owl via OWLAPI. Le programme Java vérifie le profil OWL 2 DL, la cohérence de l'ontologie et l'absence de classes insatisfiables autres que owl:Nothing. Il extrait les types et relations inférés entre individus nommés. Ce traitement ajoute 55 triplets nouveaux au graphe source.

```
Advil_400 rdf:type MedicamentAntiInflammatoire
Warfarine interagitAvec Ibuprofene
DiabeteType2 estTraitePar Glucophage_850
```

Règle de contre-indication

```
Patient(?p) ^ prend(?p, ?m) ^
aPourContreIndication(?m, ?ci) ^ CIAbsolue(?ci) ^
concerne(?ci, ?c) ^ souffreDe(?p, ?c)
  -> estContreIndiquePour(?m, ?p)
```

La règle porte sur une prise connue et une contre-indication absolue dont l'état clinique est présent chez le patient. Elle produit trois alertes : ADVIL pour P002, AMOXICILLINE BGR pour P003 et METFORMINE BGR pour P005. Le fichier SWRL est exécuté par HermiT et donne exactement le même ensemble d'alertes que R1_contre_indication.rq.

R2_interaction_specialites.rq relie deux spécialités lorsque leurs substances actives interagissent. Cette règle produit les deux directions de la relation ADVIL / COUMADINE. La requête de co-prise filtre l'ordre des IRIs pour n'afficher la paire qu'une fois. Une interaction n'est pas automatiquement une contre-indication absolue.

Monde ouvert et contrôle SHACL

OWL adopte l'hypothèse du monde ouvert. Si une référence n'est pas nommée pour un générique, la contrainte existentielle peut être satisfaite par un individu inconnu. Ce manque ne suffit donc pas à rendre l'ontologie incohérente. SHACL impose, dans le graphe de données, une référence effectivement présente, un nom, une masse positive, un code CIS valide et une source.

Mutation de test	Résultat attendu
Princeps et Generique pour le même individu	HermiT : incohérent
Deux références explicitement distinctes	HermiT : incohérent
Aucune référence nommée	HermiT : cohérent ; SHACL : violation
Âge adulte égal à 5 ou dosage négatif	SHACL : violation

SPARQL et recherche en français

Le pipeline assemble 701 triplets déclarés, 55 inférences OWL et 5 triplets de règles, soit 761 triplets dans le graphe matérialisé. Fuseki charge ce fichier explicitement. Il n'est pas supposé exécuter automatiquement la règle SWRL à chaque requête. Les fichiers Q01 à Q12 contiennent les préfixes et une limite de 200 lignes.

Requête	Résultat attendu	Construction illustrée
Q01 Catalogue	6 ligne(s)	Jointures
Q02 Patients et prises	5 ligne(s)	OPTIONAL, GROUP_CONCAT
Q03 Contre-indications justifiées	3 ligne(s)	Jointures et provenance
Q04 Interactions des prises	1 ligne(s)	FILTER, paire unique
Q05 Effets et fréquences	8 ligne(s)	Chemin de propriétés
Q06 Génériques et références	2 ligne(s)	Relation de référence
Q07 Indications	6 ligne(s)	Jointures
Q08 Composition	6 ligne(s)	Quantité et substance
Q09 Classes inférées	5 ligne(s)	VALUES et types inférés
Q10 Interactions moléculaires	2 ligne(s)	Symétrie
Q11 Comptage par principe actif	4 ligne(s)	COUNT, GROUP BY
Q12 Relation inverse	6 ligne(s)	Propriété inverse

Traduction contrôlée par spaCy

Le serveur normalise la casse et les accents, reconnaît les entités avec EntityRuler et les intentions avec Matcher. Il contrôle ensuite les mots restants, le nombre d'entités et les combinaisons autorisées. Les paramètres SPARQL sont des identifiants du corpus. Le texte de la question n'est jamais concaténé en tant que code de requête.

Exemple : « Quels médicaments sont contre-indiqués pour le patient P002 ? » identifie l'intention contre_indications et l'entité Patient_P002. Le serveur ajoute un filtre à Q03, interroge Fuseki, puis renvoie le résultat et la requête en JSON. L'interface affiche la justification et la source.

Les questions avec une négation non prise en charge, plusieurs médicaments, une entité inconnue ou une demande de traitement personnalisé sont refusées explicitement. Cette solution utilise l'API locale de spaCy, comme envisagé dans le sujet. Elle ne revendique pas une compréhension générale du français ni une API de LLM.

Résultats vérifiés et reproductibilité

Campagne finale : 51 tests réussis. Les tests ont été exécutés sous Windows, Python 3.11, Java 17 et Fuseki 5.5.0. Les versions Python exactes sont conservées dans le fichier de verrouillage. La version du serveur est choisie pour sa compatibilité avec Java 17. Le programme HermiT utilise la distribution incluse dans Owlready2 0.48.

Vérification	Preuve livrée
Profil, cohérence et satisfiabilité	validation/hermit.log
SWRL et égalité des alertes	validation/hermit-swrl.log et counts.json
Conformité des données	validation/shacl.log
51 tests, dont mutations négatives	validation/tests.log et tests.xml
RDFLib et Fuseki sur 12 SELECT	validation/fuseki-comparison.json
Faits exportés par HermiT	validation/hermit-entailed.nt
Données de comptage	validation/counts.json
Démarrage des services	validation/launch.log

La campagne couvre l'équivalence RDF/XML–Turtle, les types inférés, les alertes attendues, les contraintes de qualité, les contre-exemples OWL, les exemples français, le rejet des questions non prises en charge, l'impossibilité d'injecter du SPARQL depuis le texte libre, les réponses API, CSRF et le signalement d'une panne Fuseki.

Reconstruire après une modification

```
.venv\Scripts\python.exe -m scripts.build_ontology
.venv\Scripts\python.exe -m scripts.reason
.venv\Scripts\python.exe -m webapp.queries
.venv\Scripts\python.exe -m pytest -q --disable-warnings
```

VALIDER.cmd réalise ces étapes. Modifier le corpus JSON ou le script de schéma, puis reconstruire. La reconstruction remplace les exports générés. Redémarrer le serveur pour charger la nouvelle version. Pour inclure le test Fuseki, démarrer le serveur courant et définir RUN_FUSEKI_TESTS=1 avant pytest. Sans ce paramètre, ce test est signalé comme ignoré.

Portée de la validation

Les tests établissent des propriétés précises sur les fichiers et scénarios livrés. Ils ne constituent pas une preuve d'absence de tout défaut ni une validation médicale. Les contrôles automatiques de HermiT ont été exécutés en ligne de commande, pas au moyen de l'interface de Protégé. L'ouverture et la navigation dans la version de Protégé utilisée pour la soutenance doivent être répétées par l'équipe.

Déroulé de soutenance et préparation individuelle

Étape	Action à montrer	Durée cible
Introduction	Problème, périmètre et architecture	2 min
Ontologie	Hiérarchie, propriété et classe définie	3 min
Raisonnement	HermiT et inférences nommées	2 min
Application	P002, P004, génériques et requête visible	4 min
Qualité	Tests négatifs, sources, limites	2 min

Manipulations dans Protégé

Ouvrir pharmaonto.owl dans une fenêtre de Protégé. Choisir HermiT dans Reasoner, puis démarrer le raisonneur. Vérifier qu'aucune incohérence n'est annoncée. Examiner la hiérarchie des classes, l'expression Equivalent To de MedicamentAntiInflammatoire, puis son instance inférée Advil_400. Examiner la relation inverse estTraitePar sur DiabeteType2.

Ouvrir séparément pharmaonto-swrl.owl pour montrer la règle et les relations estContreIndiquePour déduites. Les deux fichiers partagent la même IRI et représentent deux variantes autonomes du modèle. Ne pas importer l'un dans l'autre. Selon la version de Protégé, l'onglet de règles nécessite une extension SWRL, mais la règle RDF/XML et son équivalent CONSTRUCT restent fournis.

Questions probables et réponses à maîtriser

Pourquoi une ontologie ? Pour rendre explicites les concepts, les relations et les axiomes, et obtenir des informations nouvelles par raisonnement, comme la classification d'ADVIL à partir de son principe actif.

Pourquoi HermiT et SHACL ? HermiT teste la cohérence logique. SHACL contrôle la qualité d'un graphe de données selon des exigences de présence, type et valeur. Ils répondent à deux problèmes complémentaires.

Que fait Fuseki ? Il expose le graphe via le protocole SPARQL et renvoie du JSON. Les inférences nécessaires ont déjà été matérialisées par le pipeline.

Est-ce de l'intelligence artificielle générative ? Non. Le TALN utilise spaCy avec des règles et des entités connues. Ce choix est local et reproductible, avec un périmètre de questions explicitement limité.

Pas d'alerte signifie-t-il pas de danger ? Non. Cela signifie seulement qu'aucune règle ne s'est déclenchée sur ce corpus partiel. Les conditions non saisies ne sont pas réputées absentes.

Préparation suggérée : Ahmed peut présenter le modèle et l'import, Fedi les contraintes et le raisonnement, Farah l'API et la démonstration. Cette répartition est une proposition de préparation, pas une déclaration de contributions historiques. Chaque membre doit comprendre la chaîne complète.

Sources et traçabilité

Les références ci-dessous documentent les faits pharmaceutiques sélectionnés. Consultation : **29 octobre 2026**. Les identifiants CIS permettent de retrouver la spécialité. Les fiches contiennent davantage d'informations que celles retenues dans le corpus.

ADVIL 400 mg · CIS 63646874

Fiche officielle, composition, forme et RCP. Les sections 4.1, 4.3, 4.5 et 4.8 sont utilisées selon les faits retenus.

https://base-donnees-publique.medicaments.gouv.fr/medicament/63646874/extrait

CLAMOXYL 1 g · CIS 64539416

Fiche officielle, composition, forme et RCP. Les sections 4.1, 4.3, 4.5 et 4.8 sont utilisées selon les faits retenus.

https://base-donnees-publique.medicaments.gouv.fr/medicament/64539416/extrait

GLUCOPHAGE 850 mg · CIS 67547090

Fiche officielle, composition, forme et RCP. Les sections 4.1, 4.3, 4.5 et 4.8 sont utilisées selon les faits retenus.

https://base-donnees-publique.medicaments.gouv.fr/medicament/67547090/extrait

COUMADINE 5 mg · CIS 63245753

Fiche officielle, composition, forme et RCP. Les sections 4.1, 4.3, 4.5 et 4.8 sont utilisées selon les faits retenus.

https://base-donnees-publique.medicaments.gouv.fr/medicament/63245753/extrait

AMOXICILLINE BGR 1 g · CIS 66511789

Fiche officielle, composition, forme et RCP. Les sections 4.1, 4.3, 4.5 et 4.8 sont utilisées selon les faits retenus.

https://base-donnees-publique.medicaments.gouv.fr/medicament/66511789/extrait

METFORMINE BGR 850 mg · CIS 60754492

Fiche officielle, composition, forme et RCP. Les sections 4.1, 4.3, 4.5 et 4.8 sont utilisées selon les faits retenus.

https://base-donnees-publique.medicaments.gouv.fr/medicament/60754492/extrait

Références techniques

OWL 2 Primer, W3C
https://www.w3.org/TR/owl2-primer/

SHACL, W3C
https://www.w3.org/TR/shacl/

SPARQL 1.1, W3C
https://www.w3.org/TR/sparql11-query/

Fuseki, Apache Jena
https://jena.apache.org/documentation/fuseki2/fuseki-server.html

Reconnaissance à règles, spaCy
https://spacy.io/usage/rule-based-matching

Raisonnement, Owlready2
https://owlready2.readthedocs.io/en/latest/reasoning.html